DSA0102 – C++ PROGRAMMING
ASSIGNMENT 3 (CO3)

Complete Answers • Step-by-Step Explanations • C++ Programs • Separate Outputs

Student Name: A Shiva Shankar Reddy

Register Number: 192425174

GitHub Repository: https://github.com/shivareddy1012/DSA0102-192425174-C.Plus.Plus/blob/main/Assignment-3/readme.md

Submission Date: _7-11-2026__

Compilation and GitHub Submission

Recommended compiler: g++ with C++17 support. Compile each file using:

```
g++ -std=c++17 -Wall -Wextra -pedantic Q1.cpp -o Q1
```

For memory checking, use:

```
g++ -std=c++17 -Wall -Wextra -g -fsanitize=address -fno-omit-frame-pointer Q2.cpp -o Q2
```

Run on Linux/macOS:

```
./Q1
```

Suggested repository structure:

```
DSA01-192511133B-Assignment3/\n  README.md\n  Q1/ Q1.cpp\n  Q2/ Q2.cpp\n  Q3/ Q3.cpp\n  Q4/ Q4.cpp\n  Q5/ Q5.cpp\n  Q6/ Q6.cpp\n  Q7/ Q7.cpp\n  screenshots/
```

Make at least one meaningful commit per question. Include compiler version (g++ --version), compile/run commands, output screenshots, and a short result note in README.md.

Q1 – Constructors and Constructor Overloading (6 Marks)

Answer and design

A constructor initializes an object at the moment it is created. The Book class below supports an empty record, a donated book with a title, and a fully described book. A single validating constructor is used internally so the copy-count rule is checked in one place.

```
#include <iostream>
#include <stdexcept>
#include <string>
using namespace std;

class Book {
    string title;
    int copies;
    string author;

    Book(string t, int c, string a, int) :
        title(t), copies(c < 0 ? 0 : c), author(a) {
        if (c < 0) throw invalid_argument("Copies cannot be negative");
    }

public:
    Book() : Book("", 0, "", 0) {}
    explicit Book(string t) : Book(t, 1, "Unknown", 0) {}
    Book(string t, int c) : Book(t, c, "Unknown", 0) {}
    Book(string t, int c, string a) : Book(t, c, a, 0) {}

    void display() const {
        cout << "Title: " << (title.empty() ? "(empty)" : title)
             << ", Copies: " << copies << ", Author: "
             << (author.empty() ? "(not set)" : author) << '\\n';
    }
};

int main() {
    Book emptyBook;
    Book donated("C++ Basics");
    Book limited("Data Structures", 5);
    Book full("Operating Systems", 3, "Galvin");

    emptyBook.display();
    donated.display();
    limited.display();
```

```
    full.display();

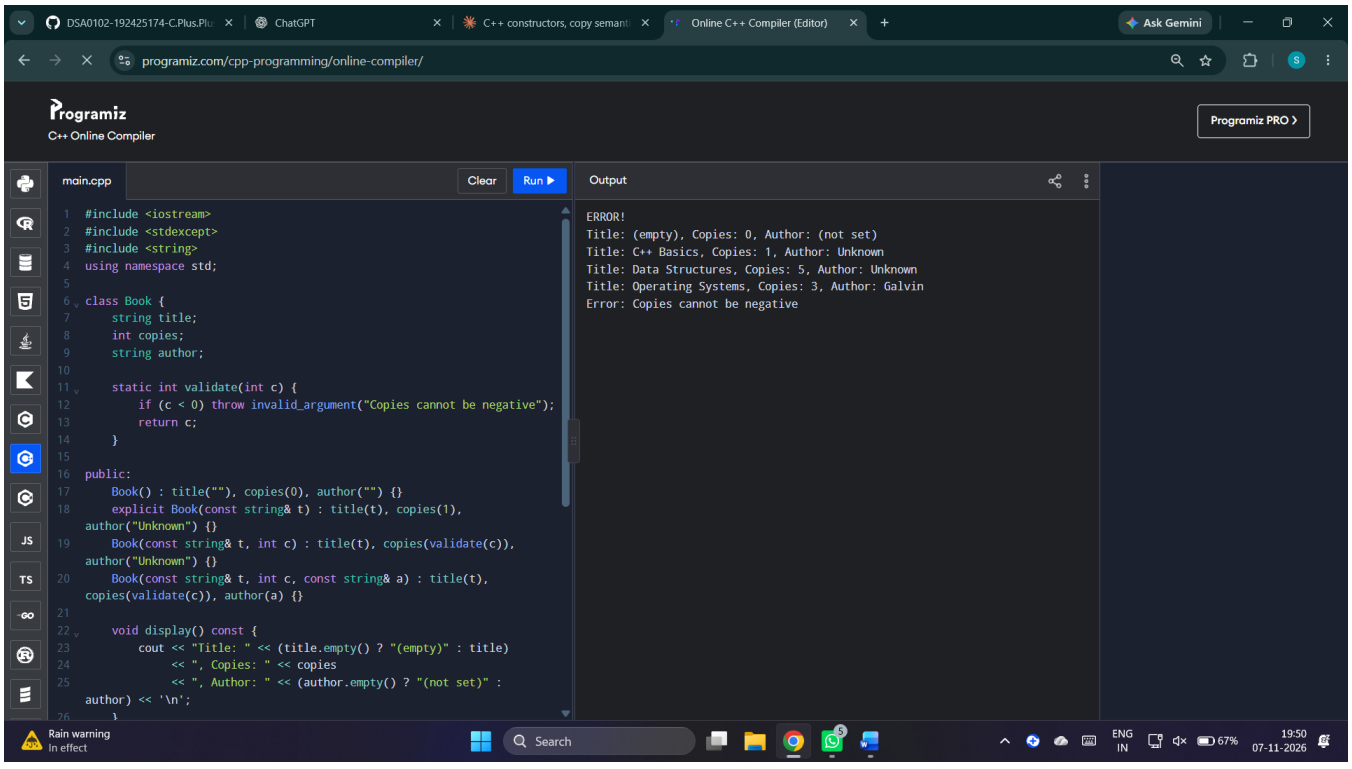
    try {
        Book invalid("Invalid Book", -2);
    } catch (const invalid_argument& e) {
        cout << "Error: " << e.what() << '\\n';
    }
}
```

Step-by-step explanation

1. title, copies and author are data members.
2. The private constructor initializes all members and rejects a negative copy count.
3. The default constructor delegates with empty values.
4. The title-only constructor represents a donated book and sets copies to 1.
5. The two-argument and three-argument constructors support more complete records.
6. display() prints the state of each object.

Expected output

```
Title: (empty), Copies: 0, Author: (not set)\nTitle: C++ Basics, Copies: 1, Author: Unknown\nTitle: Data Structures, Copies: 5, Author: Unknown\nTitle: Operating Systems, Copies: 3, Author: Galvin\nError: Copies cannot be negative
```



Design and analysis

- The compiler supplies a default constructor only when no constructor is declared by the programmer. Once a constructor is declared, the compiler does not automatically add a no-argument default constructor.

- Member initializer lists construct members directly. Assignment inside the body first default-constructs members and then assigns new values; direct initialization is generally more efficient and is required for const/reference members.
- Members are initialized in the order in which they are declared in the class, not the order written in the initializer list.
- Example states: emptyBook = ("", 0, ""); donated = ("C++ Basics", 1, "Unknown"); full = ("Operating Systems", 3, "Galvin").
- If a member is initialized in a different textual order in the initializer list, compile with -Wreorder to see a warning. Keep the initializer list in declaration order.
- If the default constructor is removed, an array declaration such as Book shelf[3]; fails because each array element needs a no-argument constructor.
- explicit prevents an unintended implicit conversion. For example, with explicit Book(string), Book b = "C++ Basics"; is rejected; Book b("C++ Basics"); is valid.

Alternative: one constructor with default arguments

```
Book(string t = "", int c = 0, string a = "") : title(t), copies(validate(c)), author(a) {}
```

Object creation is O(1) time and O(1) extra space in either design. A single constructor with defaults is shorter, but overloads can communicate the intended situations more clearly. Both designs should call one validation function so negative-copy checking is not duplicated.

Q2 – Copy Constructors: Shallow and Deep Copies (7 Marks)

Complete Code: deep copy, assignment and destructor

```
#include <iostream>
#include <string>
using namespace std;

class Prescription {
   int n;
   string* medicines;

public:
   explicit Prescription(int count)
      : n(count), medicines(count > 0 ? new string[count] : nullptr) {}

   Prescription(const Prescription& other)
      : n(other.n), medicines(n > 0 ? new string[n] : nullptr) {
      for (int i = 0; i < n; ++i) medicines[i] = other.medicines[i];
      cout << "Copy constructor called\\n";
   }

   Prescription& operator=(const Prescription& other) {
      if (this != &other) {
         string* fresh = other.n > 0 ? new string[other.n] : nullptr;
         for (int i = 0; i < other.n; ++i) fresh[i] = other.medicines[i];
         delete[] medicines;
         medicines = fresh;
         n = other.n;
      }
      return *this;
   }

   ~Prescription() { delete[] medicines; }

   void set(int i, const string& m) {
      if (i >= 0 && i < n) medicines[i] = m;
   }
   string get(int i) const {
      return (i >= 0 && i < n) ? medicines[i] : "";
   }
};

int main() {
   Prescription p(2);
```

```
    p.set(0, "Paracetamol");
    p.set(1, "ORS");

    Prescription q = p;
    q.set(1, "Cetirizine");

    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");

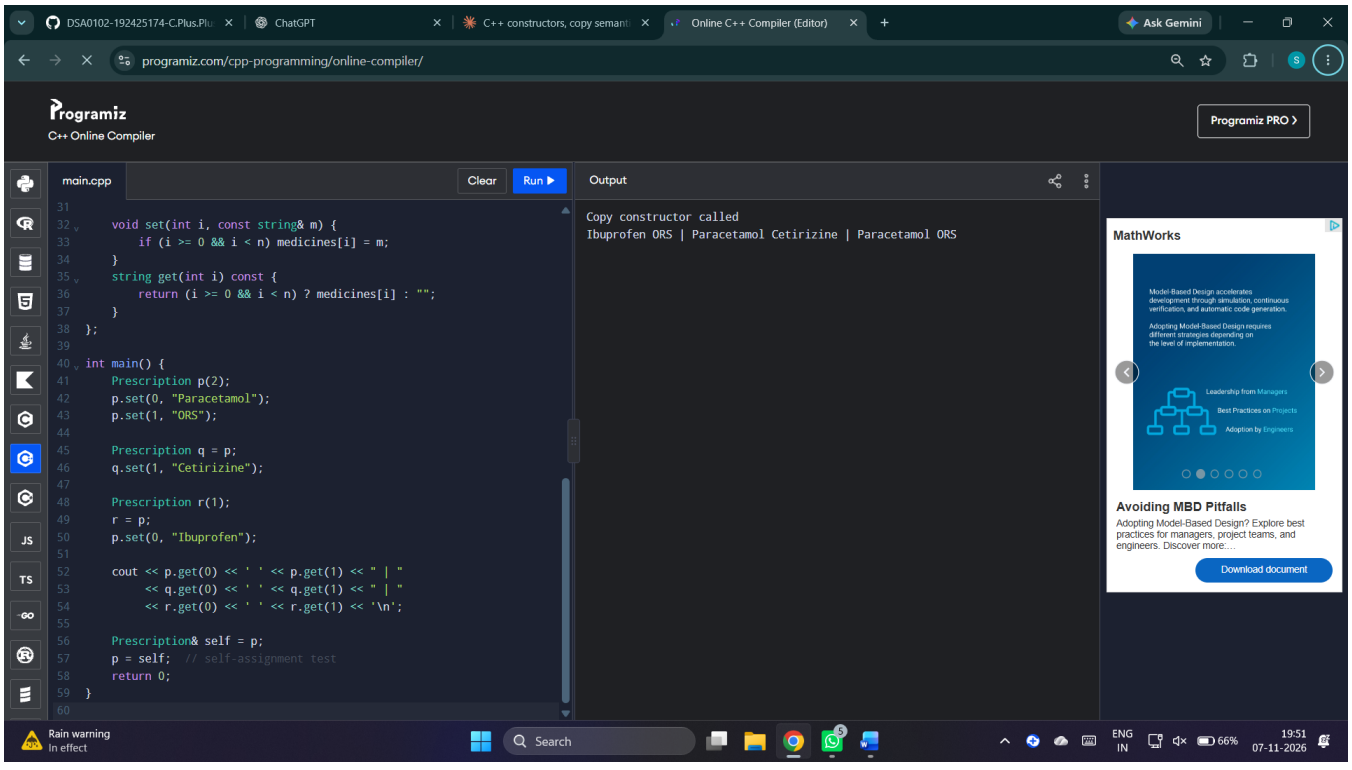
    cout << p.get(0) << ' ' << p.get(1) << " | "
         << q.get(0) << ' ' << q.get(1) << " | "
         << r.get(0) << ' ' << r.get(1) << '\\n';

    p = p; // self-assignment test
}
```

Expected output

```
Copy constructor called\nIbuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

The first line is printed by the copy constructor for q. Copy assignment for r does not print a message.



Step-by-step explanation

7. The constructor allocates an array of medicine strings.
8. The copy constructor allocates a different array and copies each string. This is a deep copy.

9. The assignment operator first checks this != &other to handle self-assignment.
10. It allocates and fills the new array before deleting the old array. This avoids losing the original data if allocation fails.
11. The destructor uses delete[] because the memory was allocated with new[].
12. Changing p does not change q or r because all three objects own separate arrays.

Memory diagrams

Shallow copy: p.medicines ──► [Paracetamol, ORS] ◄── q.medicines. Both objects point to the same array. When both destructors call delete[], the second deletion is invalid (double delete).

Deep copy: p.medicines ──► [Paracetamol, ORS] q.medicines ──► [Paracetamol, ORS]. The arrays are separate; changing one does not change the other.

Design and analysis

- Rule of Three: if a class needs a user-defined destructor, copy constructor, or copy assignment operator to manage a resource, it usually needs all three.
- The copy constructor is called when a new object is initialized from another object, when an object is passed by value, and when an object is returned by value (although copy elision may remove some copies).
- Passing by value normally makes a copy; passing by const reference does not copy the Prescription object.
- Compiler-generated copying is suitable when members already manage their own resources safely (for example std::string and std::vector). It is unsafe for a raw owning pointer that would be copied as an address.
- A shallow pointer copy is O(1), but it is not a valid independent ownership copy. A deep copy of n medicines takes O(n) time and O(n) additional space.
- With std::vector<std::string>, the compiler-generated copy is already deep for the vector elements. This follows the Rule of Zero: let standard-library resource-owning members manage memory.

Sanitizer investigation: compile with -fsanitize=address. The deep-copy version should finish without an ownership error. Removing the copy constructor causes the default memberwise copy to share the pointer; when both objects are destroyed, AddressSanitizer is expected to report an attempted double-free or heap-use-after-free depending on the exact execution. Run it locally and paste the actual sanitizer log into your report.

Copy timing investigation: test n = 10, 1,000 and 100,000. Use std::chrono::steady_clock around repeated copies, keep the result observable so the compiler cannot remove the work, and record actual measurements. Expected complexity is O(n) per deep copy; do not invent timing values.

Q3 – Dynamic Constructors and Destructors (7 Marks)

```
#include <iostream>
using namespace std;

class Event {
    int rows, seats;
    bool** booked;

public:
    Event(int r, int s) : rows(r), seats(s), booked(nullptr) {
        if (rows <= 0 || seats <= 0) {
            rows = seats = 0;
            cout << "Event created with empty seat map\\n";
            return;
        }
        booked = new bool*[rows];
        int made = 0;
        try {
            for (; made < rows; ++made) {
                booked[made] = new bool[seats]{};
            }
        } catch (...) {
            while (made > 0) delete[] booked[--made];
            delete[] booked;
            booked = nullptr;
            throw;
        }
        cout << "Event created: " << rows << " rows, " << seats << " seats\\n";
    }

    bool book(int r, int s) {
        if (!booked || r < 0 || r >= rows || s < 0 || s >= seats) return false;
        if (booked[r][s]) return false;
        booked[r][s] = true;
        return true;
    }

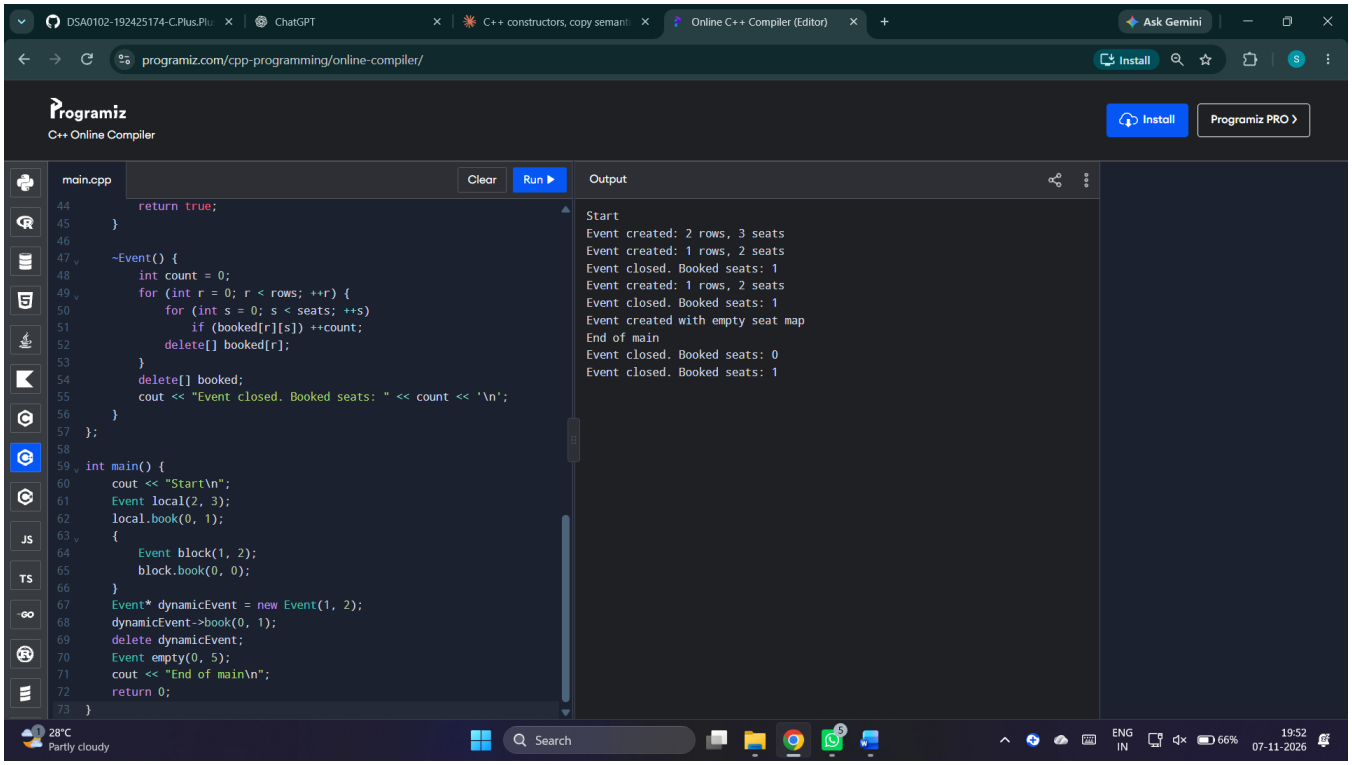
    bool cancel(int r, int s) {
        if (!booked || r < 0 || r >= rows || s < 0 || s >= seats) return false;
        if (!booked[r][s]) return false;
        booked[r][s] = false;
        return true;
    }
```

```
    ~Event() {
        int count = 0;
        for (int r = 0; r < rows; ++r) {
            for (int s = 0; s < seats; ++s) if (booked[r][s]) ++count;
            delete[] booked[r];
        }
        delete[] booked;
        cout << "Event closed. Booked seats: " << count << '\\n';
    }
};

int main() {
    cout << "Start\\n";
    Event local(2, 3);
    local.book(0, 1);
    {
        Event block(1, 2);
        block.book(0, 0);
    }
    Event* dynamicEvent = new Event(1, 2);
    dynamicEvent->book(0, 1);
    delete dynamicEvent;
    Event empty(0, 5);
    cout << "End of main\\n";
}
```

Expected output

```
Start\nEvent created: 2 rows, 3 seats\nEvent created: 1 rows, 2 seats\nEvent closed. Booked seats: 1\nEvent created: 1 rows, 2 seats\nEvent closed. Booked seats: 1\nEvent created with empty seat map\nEnd of main\nEvent closed. Booked seats: 0\nEvent closed. Booked seats: 1
```



Step-by-step explanation

13. The constructor receives the number of rows and seats and dynamically allocates an array of row pointers.
14. Each row pointer receives its own dynamically allocated bool array. Value initialization sets every seat to false.
15. book() and cancel() validate the row and seat indexes before accessing the map.
16. The destructor deletes every row with delete[] and then deletes the row-pointer array.
17. The local object is destroyed at the end of main. The block object is destroyed at the closing brace. The heap object is destroyed only when delete is called.

Design and analysis

- Automatic/local objects are destroyed in reverse order of construction when their scope ends. Block-scope objects are destroyed at the end of their block.
- A dynamically allocated object remains alive until delete is called. Forgetting delete causes a memory leak.
- Use delete[] for memory created with new[]. Using scalar delete on an array is undefined behaviour.
- A destructor has no parameters and no return type. It cannot be overloaded because an object has one destruction operation.
- Memory layout: booked points to an array of rows; each row pointer points to a separate array of seats. For rows × seats bool values, seat storage is approximately rows × seats bytes, plus row pointers and allocator overhead.
- RAII means resource acquisition is tied to object lifetime. The destructor automatically releases the resource when the object leaves scope, reducing forgotten cleanup.

Zero rows: the implementation creates an empty seat map and the destructor safely deletes a null pointer. For large maps, seat data is rows × seats bool values; for 10×20 = 200, 100×200 = 20,000 and 1,000×2,000 = 2,000,000 bool elements (implementation sizeof(bool) is normally 1 byte).

Predict the Output

```
+A +B -B copyAc use -Ac +P -P end -A
```

Explanation: a is constructed first (+A). b is constructed inside the block (+B) and destroyed at the block end (-B). use(a) passes by value, so a copy named Ac is constructed, the function prints use, and the parameter is destroyed (-Ac). The dynamically created P is constructed (+P) and destroyed by delete (-P). Finally end is printed and a is destroyed (-A).

Alternative seat-map designs

A single contiguous allocation stores rows × seats elements and accesses cell (r,s) at r × seats + s. It needs one allocation and no row-pointer array. A vector<vector<char>> is easier to manage but generally has one allocation for the outer vector plus one per row. Access is O(1) in all designs. The row-pointer design uses rows+1 allocations and extra pointer storage; contiguous storage improves locality and simplifies cleanup.

Q4 – Overloading Arithmetic and Stream Operators (7 Marks)

```
#include <iomanip>
#include <iostream>
using namespace std;

class Money {
    long long paise;
public:
    Money(long long rupees = 0, int p = 0) : paise(rupees * 100 + p) {}

    Money operator+(const Money& other) const {
        Money result;
        result.paise = paise + other.paise;
        return result;
    }
    Money operator-(const Money& other) const {
        Money result;
        result.paise = paise - other.paise;
        return result;
    }
    Money& operator+=(const Money& other) {
        paise += other.paise;
        return *this;
    }
    bool operator==(const Money& other) const { return paise == other.paise; }
    bool operator<(const Money& other) const { return paise < other.paise; }

    friend ostream& operator<<(ostream& out, const Money& m) {
        long long absValue = m.paise < 0 ? -m.paise : m.paise;
        if (m.paise < 0) out << "-";
        out << "Rs " << absValue / 100 << '.' << setw(2) << setfill('0')
            << absValue % 100 << setfill(' ');
        return out;
    }
    friend istream& operator>>(istream& in, Money& m) {
        long long rupees; int p;
        if (in >> rupees >> p) m.paise = rupees * 100 + p;
        return in;
    }
};

int main() {
    Money a(120, 75), b(79, 25);
    Money c = a + b;
```

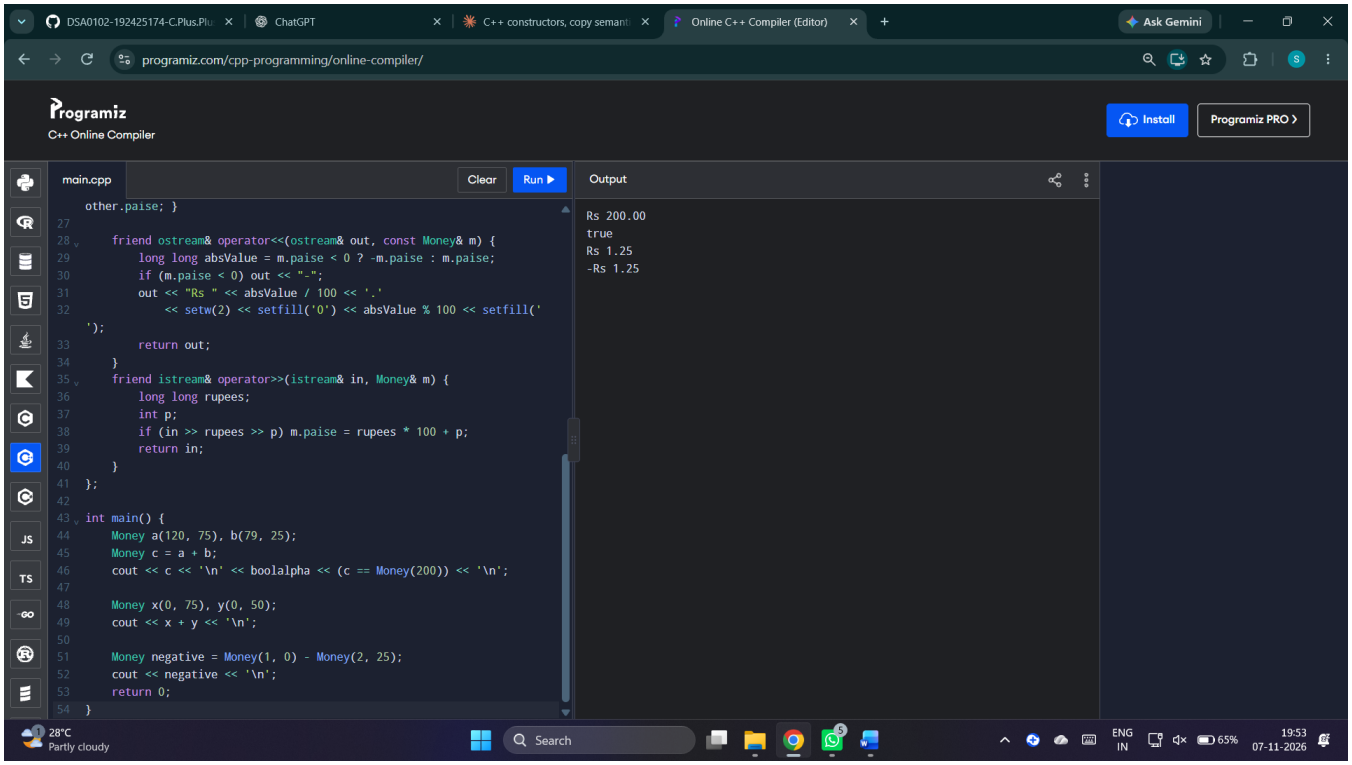
```
    cout << c << '\\n' << boolalpha << (c == Money(200)) << '\\n';

    Money x(0, 75), y(0, 50);
    cout << x + y << '\\n';

    Money negative = Money(1, 0) - Money(2, 25);
    cout << negative << '\\n';
}
```

Expected output

```
Rs 200.00\ntrue\nRs 1.25\n-Rs 1.25
```



Step-by-step explanation

18. Money stores one integer: total paise. This avoids floating-point rounding errors for ordinary currency arithmetic.
19. The constructor converts rupees and paise into a single paise count.
20. + returns a new Money object and does not change either operand.
21. += changes the left operand and returns *this by reference so chained updates are possible.
22. == and < compare the integer paise values.
23. << is a non-member because the left operand is ostream, not Money. It is a friend so it can read the private paise member.
24. The expression a + b is equivalent to a.operator+(b) for this member operator.

Design decisions and evaluation

- Multiplication of two Money values is usually not meaningful: rupees × rupees does not produce a currency amount. Multiplying Money by a quantity or scalar may make sense.
- The example represents a negative balance by printing a leading minus sign. A real billing system may instead reject negative bills or model refunds explicitly.
- Each arithmetic/comparison operation is O(1) time and O(1) extra space.
- With member operators, a + b works when the left operand is Money. A non-member overload can support conversions on either side, such as 100 + m and m + 100, if suitable constructors are provided.
- Named functions add(a,b) and equals(a,b) are also O(1), but operators make arithmetic expressions easier to read.

For the required one-million-addition comparison, write a loop for the integer-paise Money version and a double version, use std::chrono, compile once with -O0 and once with -O2, and record the measured durations. The assignment does not provide benchmark values.

Q5 – Overloading Increment, Comparison and Subscript Operators (6 Marks)

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

class Time {
   int seconds;
public:
   explicit Time(int s = 0) : seconds(s) {}
   Time& operator++() { ++seconds; return *this; }       // prefix
   Time operator++(int) { Time old(*this); ++seconds; return old; } // postfix
   bool operator==(const Time& t) const { return seconds == t.seconds; }
   bool operator<(const Time& t) const { return seconds < t.seconds; }
   int value() const { return seconds; }
};

class Playlist {
   vector<string> tracks;
public:
   Playlist(initializer_list<string> list) : tracks(list) {}
   string& operator[](int i) {
      if (i < 0 || i >= static_cast<int>(tracks.size()))
         throw out_of_range("Playlist index out of range");
      return tracks[i];
   }
   const string& operator[](int i) const {
      if (i < 0 || i >= static_cast<int>(tracks.size()))
         throw out_of_range("Playlist index out of range");
      return tracks[i];
   }
   int size() const { return static_cast<int>(tracks.size()); }
};

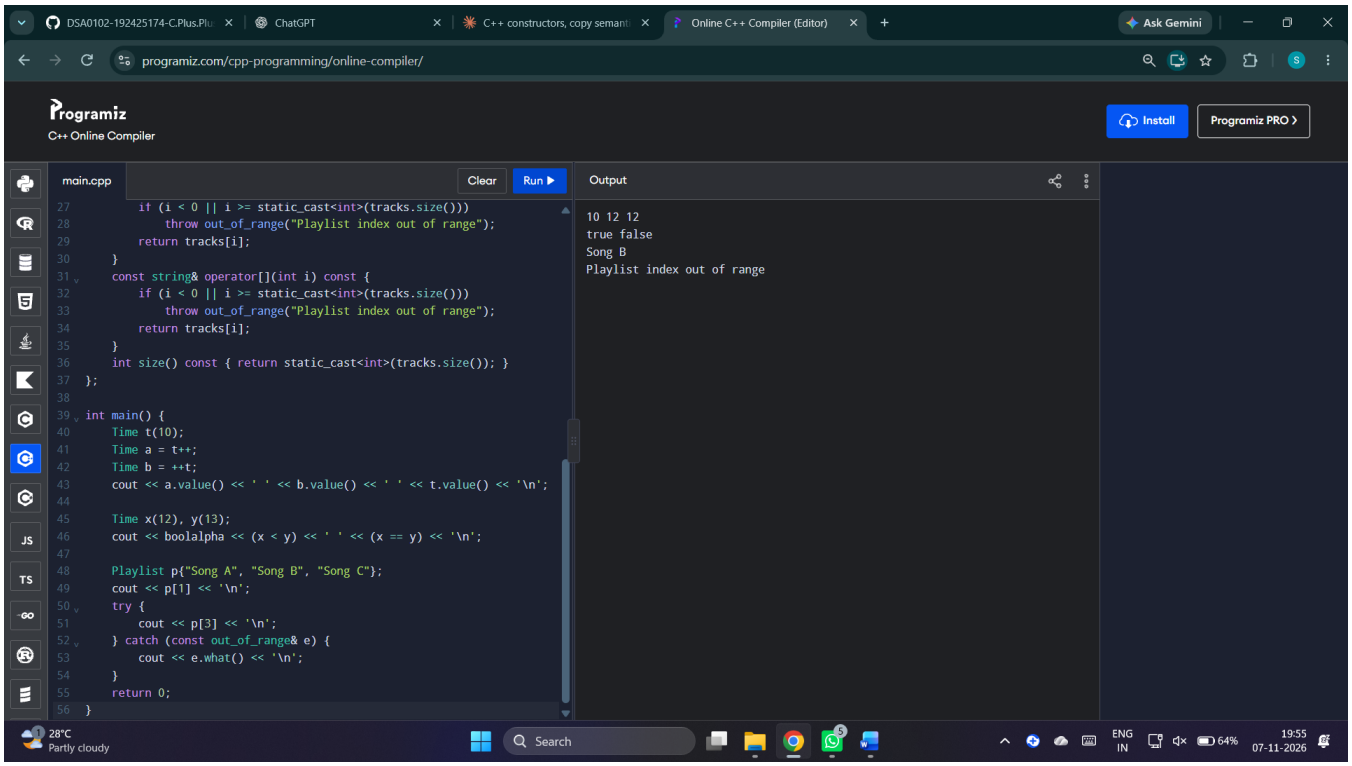
int main() {
   Time t(10);
   Time a = t++;
   Time b = ++t;
   cout << a.value() << ' ' << b.value() << ' ' << t.value() << '\\n';

   Time x(12), y(13);
   cout << boolalpha << (x < y) << ' ' << (x == y) << '\\n';
```

```
  Playlist p{"Song A", "Song B", "Song C"};
  cout << p[1] << '\\n';
  try { cout << p[3] << '\\n'; }
  catch (const out_of_range& e) { cout << e.what() << '\\n'; }
}
```

Expected output

```
10 12 12\ntrue false\nSong B\nPlaylist index out of range
```



Step-by-step explanation

25. Prefix ++ increments the object and returns a reference to the updated object.
26. Postfix ++ has an unused int parameter to distinguish it from prefix. It saves and returns the old value, so it creates a copy.
27. In a = t++, a receives the old value. In b = ++t, b receives the new value.
28. The two operator[] overloads allow both modification through a non-const Playlist and read-only access through a const Playlist.
29. The range check rejects negative indexes and indexes greater than or equal to size.

Design and analysis

- The int dummy parameter is the language convention that distinguishes postfix from prefix.
- Postfix returns a copy because its result must represent the value before increment. Prefix can return a reference and avoid that copy.
- Out-of-range access is handled by throwing std::out_of_range. This is safer than reading invalid memory.

- The supplied Counter prediction program prints: 5 7 7. c++ first returns old value 5 to a; then prefix increments c to 7 and returns it to b.
- For Time measured in seconds, rollover formatting can display seconds as hh:mm:ss. For 59:59 plus one second, normalize to 1:00:00.
- Many programmers prefer ++i for class types because it usually avoids creating the old-value temporary. Both are O(1) for this simple type.

The checked and unchecked playlist accesses are O(1). A checked access performs an additional comparison. Measure 10,000,000 accesses using a timed loop; use the same data and compiler flags for a fair comparison. The assignment requires measured results, so insert your actual timings.

Q6 – The Rules of Operator Overloading (7 Marks)

```
#include <iostream>
#include <cstring>
#include <stdexcept>
using namespace std;

class Text {
   char* data;
public:
   Text(const char* s = "") {
      if (!s) s = "";
      data = new char[strlen(s) + 1];
      strcpy(data, s);
   }
   Text(const Text& other) {
      data = new char[strlen(other.data) + 1];
      strcpy(data, other.data);
   }
   Text& operator=(const Text& other) {
      if (this != &other) {
         char* fresh = new char[strlen(other.data) + 1];
         strcpy(fresh, other.data);
         delete[] data;
         data = fresh;
      }
      return *this;
   }
   ~Text() { delete[] data; }

   Text operator+(const Text& other) const {
      size_t n = strlen(data) + strlen(other.data);
      char* temp = new char[n + 1];
      strcpy(temp, data);
      strcat(temp, other.data);
      Text result(temp);
      delete[] temp;
      return result;
   }
   bool operator==(const Text& other) const {
      return strcmp(data, other.data) == 0;
   }
   bool operator!=(const Text& other) const { return !(*this == other); }

   friend ostream& operator<<(ostream& out, const Text& t) {
```

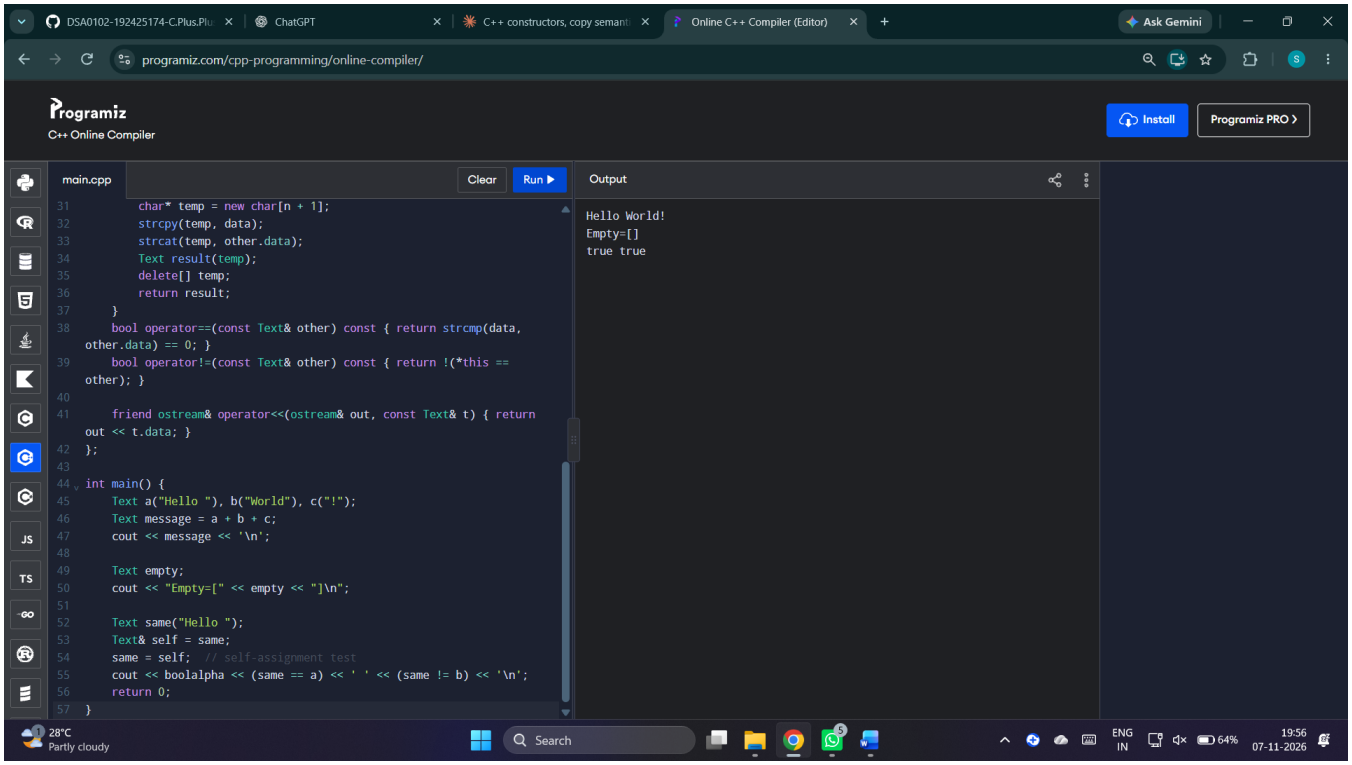
```
        return out << t.data;
    }
};

int main() {
    Text a("Hello "), b("World"), c("!");
    Text message = a + b + c;
    cout << message << '\\n';

    Text empty;
    cout << "Empty=[" << empty << "]\\n";
    Text same("Hello ");
    same = same;
    cout << boolalpha << (same == a) << ' ' << (same != b) << '\\n';
}
```

Expected output

```
Hello World!\nEmpty=[]\ntrue true
```



Step-by-step explanation

30. The constructor allocates enough memory for the characters and the terminating null character.
31. The copy constructor allocates a separate character array and copies the text.
32. The assignment operator checks self-assignment, creates the replacement array, copies the characters, deletes the old array and returns *this.
33. The destructor releases the array with delete[].
34. + creates a new combined string. == and != compare the character contents.

35. << is a friend non-member because the left operand is an output stream.

Rules and analysis

- Operators that cannot be overloaded include ::, ., .*, ?: and sizeof.
- Overloading cannot change an operator's precedence, associativity or number of operands (arity). At least one operand must be a user-defined type.
- operator= must be a member function.
- A friend is useful when a non-member operator needs access to private data, especially stream insertion where the left operand is ostream.
- The expression a + b + c is evaluated as (a + b) + c for the usual left-associative +. Temporary Text objects may be created; copy elision and move operations can affect the exact copy count.
- For strings with lengths n1, n2 and n3, a straightforward a+b+c may copy approximately n1+n2 characters for the first result and then n1+n2+n3 for the second result: O(n1+n2+n3) time, with intermediate temporary storage.
- Using std::string is preferred in normal applications because it already manages memory, copying, assignment and concatenation safely. A custom Text class is useful for learning ownership or when a specialized representation is genuinely required.

Run the class under AddressSanitizer, test self-assignment and empty/long messages. Compare 100,000 concatenations against std::string using identical inputs and compiler flags; record the measured time. Do not claim a timing result without running it.

Q7 – Design a Complex-Number Class from First Principles (10 Marks)

Stage 1 – Interpret the problem

A complex impedance contains a real part and an imaginary part. Required data: real value, imaginary value, and operations +, -, *, /, ==, printing and magnitude. Constructors: default (0+0j), real-only (R+0j), and real-plus-imaginary.

Stage 2 – Design and formulas

- Addition: (a+jb)+(c+jd) = (a+c)+j(b+d).
- Subtraction: (a+jb)-(c+jd) = (a-c)+j(b-d).
- Multiplication: (a+jb)(c+jd) = (ac-bd)+j(ad+bc).
- Division: ((a+jb)/(c+jd)) = ((ac+bd)+j(bc-ad))/(c^2+d^2). Reject a zero denominator.
- Magnitude: |a+jb| = sqrt(a^2+b^2).
- Arithmetic and equality can be members. Stream insertion is a friend non-member because ostream is the left operand.

Stage 3 – Complete C++ implementation

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <stdexcept>
using namespace std;

class Complex {
   double re, im;
public:
   Complex() : re(0), im(0) {}
   Complex(double real) : re(real), im(0) {}
   Complex(double real, double imag) : re(real), im(imag) {}

   Complex operator+(const Complex& z) const {
      return Complex(re + z.re, im + z.im);
   }
   Complex operator-(const Complex& z) const {
      return Complex(re - z.re, im - z.im);
   }
   Complex operator*(const Complex& z) const {
      return Complex(re*z.re - im*z.im, re*z.im + im*z.re);
   }
   Complex operator/(const Complex& z) const {
      double den = z.re*z.re + z.im*z.im;
      if (den == 0.0) throw domain_error("Division by zero impedance");
      return Complex((re*z.re + im*z.im)/den,
                     (im*z.re - re*z.im)/den);
```

```
    }
    bool operator==(const Complex& z) const {
        const double eps = 1e-9;
        return fabs(re-z.re) < eps && fabs(im-z.im) < eps;
    }
    double magnitude() const { return sqrt(re*re + im*im); }

    friend ostream& operator<<(ostream& out, const Complex& z) {
        out << fixed << setprecision(2) << z.re;
        if (z.im >= 0) out << " + j" << z.im;
        else out << " - j" << -z.im;
        return out;
    }
};

int main() {
    const double pi = acos(-1.0);
    double frequency = 50.0;
    double omega = 2.0 * pi * frequency;
    double R = 10.0;
    double L = 0.01;
    double C = 100e-6;

    Complex zr(R, 0);
    Complex zl(0, omega * L);
    Complex zc(0, -1.0 / (omega * C));

    Complex series = zr + zl + zc;
    Complex parallel = (zr * zl) / (zr + zl);

    cout << "Resistor: " << zr << '\\n';
    cout << "Inductor: " << zl << '\\n';
    cout << "Capacitor: " << zc << '\\n';
    cout << "Series impedance: " << series << '\\n';
    cout << "Series magnitude: " << series.magnitude() << '\\n';
    cout << "Parallel (R and L): " << parallel << '\\n';

    try {
        Complex zero;
        Complex invalid = zr / zero;
        cout << invalid << '\\n';
    } catch (const domain_error& e) {
        cout << "Error: " << e.what() << '\\n';
```

```
    }
}
```

Expected output (rounded to 2 decimal places)

```
Resistor: 10.00 + j0.00\nInductor: 0.00 + j3.14\nCapacitor: 0.00 - j31.83\nSeries impedance: 10.00 - j28.69\nSeries magnitude: 30.38\nParallel (R and L): 0.90 + j2.87\nError: Division by zero impedance
```

